DevOps Lab - Command Sheet

Commands in the order you run them. Lines starting with # are comments (not typed). Replace <...> values with your own.

Exp 1 - Git Essentials

1. Install check and identity

```
git --version
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --list
```

2. Local repository and first commit

```
mkdir git-lab
cd git-lab
# create index.html
git init
git status
git add index.html            # or: git add .
git status
git commit -m "Initial commit of the project"
git log
git log --oneline
```

3. Connect to GitHub and push

```
git branch -M main
git remote add origin https://github.com/<user>/git-lab.git
git remote -v
git push -u origin main
```

4. Branch and merge

```
git checkout -b feature/new-feature
# edit index.html
git add .
git commit -m "Add new feature"
git branch
git checkout main
git merge feature/new-feature
git push origin main
git branch -d feature/new-feature
```

5. Pull request flow

```
git checkout -b feature/pr-demo
# edit a file
git add .
git commit -m "PR demo"
git push -u origin feature/pr-demo
# GitHub: Compare & pull request -> Create pull request -> Merge pull request -> Confirm merge
git checkout main
git pull origin main
```

6. Change visibility (GitHub website, no commands)

```
Repo > Settings > Danger Zone > Change visibility > select option
> type the repo name > I understand, change visibility
```

Exp 2 - Flask Docker Deployment

1. Check and prepare

```
docker --version
docker info
mkdir flask-docker-app
cd flask-docker-app
# create app.py, requirements.txt, Dockerfile
```

2. Build and run locally

```
docker build -t flask-docker-app .
docker images
docker run -p 5000:5000 flask-docker-app
# browse http://localhost:5000   (Ctrl+C to stop)
docker run -p 5050:5000 flask-docker-app     # if 5000 is busy
```

3. Background mode (optional)

```
docker run -d --name flask-web -p 5000:5000 flask-docker-app
docker ps
docker ps -a
docker logs flask-web
docker stop flask-web
docker rm flask-web
```

4. Push to Docker Hub

```
docker login
docker tag flask-docker-app klassy01/flask-docker-app:latest
docker images
docker push klassy01/flask-docker-app:latest
```

5. Pull and run to test

```
docker pull klassy01/flask-docker-app:latest
docker run -p 5050:5000 klassy01/flask-docker-app:latest
# browse http://localhost:5050
```

6. Clean up

```
docker ps -a
docker rm <container_id>
docker rmi flask-docker-app
```

Exp 3 - CI Pipeline - GitHub Actions + Docker Hub

1. Project setup

```
mkdir python-ci-docker-lab
cd python-ci-docker-lab
mkdir tests
mkdir -p .github/workflows
# create app.py, tests/test_app.py, requirements.txt, Dockerfile, .gitignore
# save workflow as .github/workflows/ci-dockerhub.yml
```

2. Local test

```
python app.py
pytest -q
docker build -t klassy01/python-ci-lab:local .
docker run --rm klassy01/python-ci-lab:local
```

3. Push to GitHub (triggers the workflow)

```
# first: add secrets DOCKERHUB_USERNAME and DOCKERHUB_TOKEN in
# repo Settings > Secrets and variables > Actions
git init
git add .
git commit -m "init lab"
git branch -M main
git remote add origin https://github.com/<your-username>/python-ci-docker-lab.git
git push -u origin main
# watch: GitHub > Actions tab
```

4. Release tag (optional)

```
git tag v1.0.0
git push origin v1.0.0
```

5. Verify the image

```
docker pull klassy01/python-ci-lab:latest
docker run --rm klassy01/python-ci-lab:latest
docker images klassy01/python-ci-lab
```

Exp 4 - Jenkins Deployment Using Docker

1. Setup

```
docker --version
docker compose version
mkdir ~/jenkins-static-site && cd ~/jenkins-static-site
# PowerShell: mkdir jenkins-static-site ; cd jenkins-static-site
# create docker-compose.yml
```

2. Start Jenkins

```
docker compose up -d
docker ps
docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
docker compose logs jenkins          # alternative place to find the password
```

3. In the browser (no commands)

```
http://localhost:8080 > paste password > Install suggested plugins > create admin user
Manage Jenkins > Plugins > Available plugins > HTML Publisher > Install
New Item > karunya-site-pipeline > Pipeline > Pipeline script > paste Jenkinsfile > Save
Build Now > open 'Karunya University - Simple Site' link
```

4. Clean up

```
docker compose down
```

Exp 6 - Manage Kubernetes Resources Using CLI

1. Install and start

```
docker --version
choco install kubernetes-cli          # PowerShell as Administrator
kubectl version --client
choco install minikube
minikube start --driver=docker
minikube status
kubectl get nodes
```

2. Pod

```
kubectl apply -f nginx-pod.yaml
kubectl get pods
kubectl describe pod nginx-pod
kubectl port-forward pod/nginx-pod 8080:80
# browse http://localhost:8080   (Ctrl+C to stop)
kubectl delete pod nginx-pod
```

3. Deployment and scaling

```
kubectl create deployment my-nginx --image=nginx
kubectl get deployments
kubectl get pods -l app=my-nginx
kubectl scale deployment my-nginx --replicas=3
kubectl get pods
```

4. Rolling update and rollback

```
kubectl set image deployment/my-nginx nginx=nginx:1.25
kubectl rollout status deployment/my-nginx
kubectl rollout history deployment/my-nginx
kubectl rollout undo deployment/my-nginx
```

5. Service

```
kubectl expose deployment my-nginx --type=NodePort --port=80
kubectl get svc
minikube service my-nginx --url
kubectl delete svc my-nginx
```

6. Clean up

```
kubectl delete deployment my-nginx
kubectl delete svc my-nginx
kubectl delete all --all -n default
minikube stop
minikube delete
```

Exp 7 - Kubernetes Deployment and Service for Python App

1. Cluster and image

```
minikube start --driver=docker
minikube status
kubectl get nodes
docker pull klassy01/flask-docker-app
```

2. Deployment

```
# create python-deployment.yaml
kubectl apply -f python-deployment.yaml
kubectl get deployments
kubectl get pods
kubectl describe deployment python-web-deployment
```

3. Service and access

```
# create python-service.yaml
kubectl apply -f python-service.yaml
kubectl get svc
minikube service python-web-service --url
# open the printed URL (keep the terminal open)
```

4. Clean up

```
kubectl delete svc python-web-service
kubectl delete deployment python-web-deployment
kubectl get all
minikube stop
minikube delete
```

If you use the record's image instead, pull dockersamples/flaskapp and put it in the Deployment YAML.